

Architecture

01 — Architecture

Last updated: 2026-06-08

1. Overview

Productivity Shastra is a **single-page React application** that bundles one "shell" app and several independent productivity **tools** into one deployable frontend. It is currently **frontend-only** — all data persists to the browser's `localStorage`. A backend (API + database) is planned but intentionally not yet built.

2. Repository layout (monorepo)

```
APPPP/  
├─ frontend/      ← the merged React app (everything users see)  
├─ backend/       ← reserved for the future API/server (empty scaffold)  
├─ documentation/ ← these official docs  
├─ CLAUDE.md      ← project handoff (auto-loaded by Claude Code)  
└─ .claude/       ← Claude memory anchor (do not move)
```

The original standalone tool folders (`Power Planner/` , `Reasons Eliminator/` , `Meeting Success Maximizer/` , `Time Finder/` , plus the original `Base+Time Auditor/`) are the **pre-merge sources**. They are kept only until the merged app is verified in the browser, then removed.

3. Frontend architecture

3.1 The shell

The base app (originally "Base + Time Auditor", package name now `productivity-shastra`) provides:

- **Auth** — a frontend-only mock (`src/context/AuthContext.jsx`). Supabase was removed (`src/lib/supabase.js` exports `isConfigured = false`), so login runs in "demo/offline mode" against `localStorage` (`ta_user`). **No backend is required to run the app today.**
- **Routing** — `react-router-dom v6`, defined in `src/routes/AppRoutes.jsx` , guarded by `src/routes/ProtectedRoute.jsx` .
- **Layouts** — `HomeLayout` (top navbar, no sidebar — the dashboard/hub), `DashboardLayout` (sidebar — main app pages), `AuthLayout` (login).
- **The hub** — `src/pages/Home.jsx` renders the module-picker cards that link to each tool.
- **Time Auditor** — the shell's own native tool (`/dashboard` , `/time-auditor`).

3.2 The tools (merged features)

Each tool was copied verbatim into `frontend/src/features/<tool>/` and mounted at a route the shell already reserved:

Tool	Feature folder	Route	Internal routing
Power Planner	features/power-planner/	/power-planner	single page
Reasons Eliminator	features/reason-eliminator/	/reason-eliminator/*	~17 nested routes
Meeting Success Maximizer	features/meeting/	/meeting-framework/*	5 nested routes
Time Finder	features/time-finder/	/time-finder/*	~18 nested routes

Each tool keeps **its own logic, components, layout and localStorage data** untouched. The merge only changed three things per tool:

1. **Router ownership.** Each tool's standalone `<BrowserRouter>` was removed; the shell owns the single router. Each tool's `App.jsx` is now a router-free feature component whose internal `<Route>` paths are **relative** to its mount point.
2. **Internal navigation prefixing.** Tools that used bare absolute paths (`navigate('/dashboard')`) were prefixed to their mount (`/time-finder/...` , `/meeting-framework/...`). Reasons Eliminator and Power Planner were already namespaced and needed (almost) none.
3. **Style isolation** (see §4).

3.3 Mount pattern

The shell's `src/pages/<Tool>Page.jsx` is a thin wrapper that:

- imports the tool's feature root (`features/<tool>/App.jsx`),
- wraps it in a full-screen `min-h-screen` container carrying **the tool's own light theme** (so it doesn't inherit the shell's dark theme),
- renders a small fixed `<HubLink/>` (`src/components/ui/HubLink.jsx`) back to `/home` .

Tools are mounted **outside** `HomeLayout` (full-screen, like the shell's own Time Auditor) because each tool ships its own sidebar/header and should not be double-wrapped.

4. Styling isolation (Tailwind)

There is **one** Tailwind config (`frontend/tailwind.config.js`) and **one** global stylesheet (`frontend/src/index.css` , dark theme). To stop the five independent themes from clashing:

- **Brand palettes were deep-merged.** The shell's numeric `brand-50...950` scale and the tools' named keys (`brand-red` , `brand-gray-*` , `brand-soft` , etc.) coexist on one `brand` object.
- **Conflicting tokens were namespaced.** Meeting's `ink` → `mkink` ; Time Finder's `ink-*` → `tfink-*` . (Class usages were rewritten accordingly.)
- **Global body resets were dropped.** Each tool's original `index.css` set a light `body` background that would fight the dark shell. Instead, each tool's page wrapper sets its own background/text, so the tool renders its light look on a self-contained surface. Component-only styles a tool depends on live in a `features/<tool>/scoped.css` (Power Planner, Reasons Eliminator).

5. State & persistence

All state is client-side. `localStorage` keys are namespaced per tool, so no tool can corrupt another's data:

Tool	Key prefix
Shell (Time Auditor / auth)	<code>ta_*</code> , <code>ps_*</code>
Power Planner	<code>power-planner-*</code>
Reasons Eliminator	<code>altus.reasonEliminator.*</code>
Meeting	<code>msm_*</code>
Time Finder	<code>assessments</code> , <code>currentAssessment</code> , ... (generic — see TRD risk note)

6. Build & deploy

- **Tooling:** Vite 5, single build → `frontend/dist/` .
- **Static hosting today:** the built `dist/` is a static SPA; deployable to any static host (Netlify/Vercel/S3/Nginx). SPA fallback to `index.html` required.
- **Future:** when the backend lands, add an API base URL via env and a reverse proxy or same-origin `/api` .

See [TRD](#) for concrete versions and conventions, and [Backend Schema](#) for the planned server.